

1. Tutorial: construcción de un libro de cálculo con el DSL

[Te propongo que no pongas en el título que es para un libro de cálculo. debería ser para definir una visualización de horarios, y casi *de casualidad* es que lo vamos a exportar a un libro de cálculo, pero no tiene por qué ser así.]

Este capítulo narra, paso a paso, el proceso de construir un programa completo en el DSL a partir de la descripción de un problema de horario. **NUEVO PÁRRAFO.**

El ejemplo **será(es)** un **horario de defensas de tesis**: dado un conjunto de profesores, estudiantes y locales, se necesita **una aplicación**(~~un libro de cálculo~~) que liste las defensas programadas, (**compute**¹) automáticamente cuántas defensas acumula cada profesor y resalte en rojo a los profesores con conflicto de horario.

La filosofía del sistema es (**separar la descripción del problema de su generación**²) : el programador declara la estructura **de la aplicación**(~~del libro~~) (tablas, fórmulas, coloreado condicional) (y el generador produce el archivo **.xlsx** de forma automática.³)

Pon una estructura de las secciones.

1: calcule

2: justamente por esto te propongo no hacer referencias al libro de cálculo.

3: Y esto ponlo como una de las opciones.

1.1 Requisitos

[Cosa empezando con subcosa.]

1.1.1 Software necesario

Para utilizar el generador se necesitan dos programas:

Steel Bank Common Lisp (SBCL) Intérprete de Common Lisp. Dis-

ponible en <https://www.sbcl.org/>. Versión mínima recomendada: 2.0.

Python 3.10 o superior Para ejecutar el código generado. Además de Python estándar, se requiere la biblioteca **openpyxl**:

```
pip install openpyxl
```

1.1.2 Estructura del proyecto

El código fuente del generador se organiza en los siguientes archivos:

```
generador-excel/  
+-- dsl-directo.lisp          <- DSL y macros (no  
    modificar)  
+-- ast-def.lisp             <- definicion de nodos  
    AST (no modificar)  
+-- generate-code-direct.lisp <- backend de  
    generacion (no modificar)  
+-- code-generation-utils.lisp  
+-- hoja_con_formulas.py     <- backend Python (no  
    modificar)  
\-- mi-horario/  
    +-- data-tutorial.lisp    <- datos concretos (el  
        usuario crea este)  
    \-- tutorial-horario.lisp <- programa DSL (el  
        usuario crea este)
```

Los archivos de la raíz son la **biblioteca del generador** y no deben modificarse. El usuario únicamente crea su propio programa DSL y su archivo de datos.

1.1.3 Ejecución

[Esto déjalo para el final... en este momento del tutorial crea más ruido de lo que aclara. Eso vamos a ponerlo cuando ya tengamos todos los ficheros listos.]

Hay dos formas equivalentes de ejecutar el flujo completo.

Opción A — **script incluido** (recomendada para comenzar):

```
cd tutorial/  
chmod +x generar.sh    # solo la primera vez
```

```
./generar.sh
```

Opción B — carga directa con SBCL:

```
cd tutorial/  
sbcl --script tutorial-horario.lisp
```

Ambas opciones cargan el DSL, construyen el árbol AST, generan el programa de construcción del libro y lo ejecutan para crear el archivo `.xlsx`. La diferencia es que el script imprime mensajes de progreso y verifica que el archivo fue creado correctamente. (La sección 1.8 explica el flujo con más detalle.⁴)

4: Perfecto... entonces vamos a dejarlo para allá, para esa sección.

[TRANSICIÓN A LO QUE SIGUE.](#)

1.2 Conceptos fundamentales

1.2.1 El árbol de sintaxis abstracta (AST)

Cuando el usuario escribe definiciones en Common Lisp usando las macros (del sistema⁵), estas construyen automáticamente objetos en memoria que forman un **árbol de sintaxis abstracta (AST)**. Cada nodo del árbol representa un concepto del problema: una tabla, una expresión de celda, un cuantificador existencial, una regla de coloreado.

5: ¿de qué sistema?

(El usuario **nunca manipula el árbol directamente**: trabaja con las macros de alto nivel descritas en este documento.⁶)

6: ¿por qué esto es relevante?

[TRANSICIÓN A LO QUE SIGUE.](#)

1.2.2 El DSL es independiente del generador de código

El AST no contiene ninguna referencia a Excel, a Python (ni a ninguna otra tecnología concreta⁷). Es una representación **neutral del problema**. (Cualquier componente que sepa recorrer el árbol^{Q1}) e (interpretar sus nodos^{Q2}) puede actuar (como generador de código^{Q3}) (para el destino que desee^{Q4}).

7: ¿tecnologías de qué?

Q1: ¿Qué es esto?

Q2: ¿Qué es esto?

Q3: ¿Qué es esto?

Q4: ¿Qué es esto?

En este trabajo el generador implementado toma el AST y produce un **script Python** que usa la biblioteca `openpyxl` para construir **un(él)** archivo `.xlsx`. Pero el mismo DSL podría alimentar, sin cambio alguno en el programa del usuario, un generador diferente que produjera, por ejemplo,

una hoja LibreOffice, un informe PDF o una tabla HTML.

El generador implementado en este trabajo

El backend concreto que se distribuye con el DSL genera un script Python que usa `openpyxl` para construir archivos Excel (`.xlsx`). Las fórmulas de celda, los datos y las reglas de formato condicional quedan incrustados en el archivo y funcionan en cualquier versión de Excel o LibreOffice Calc sin necesidad de tener instalado Python ni SBCL.

1.2.3 Etapas de la canalización

[¿Qué es una canalización?]

El proceso de generación tiene tres etapas (**elaramente**) separadas:

1. **Construcción del AST.** (~~textbfExpansion de macros.~~) Cada (forma del DSL^{Q5}) (`def-table`, `exists`, `conditional-rendering`) (se expande^{Q6}) en un nodo del AST.
2. **Generación de código.** (El árbol⁸) se pasa al generador, que recorre cada nodo y produce (el programa de construcción del libro⁹). En (el backend incluido¹⁰), ese programa es un script Python.
3. **Ejecución.** El programa generado se ejecuta y crea el archivo de salida con datos, fórmulas y reglas de formato condicional incrustadas.

Q5: ¿Qué es esto?

Q6: ¿Qué es esto?

8: ¿qué árbol?

9: Sería de la aplicación de horarios en el formato deseado, ¿no?

10: ¿incluido dónde o en qué?

TRANSICIÓN A LO QUE SIGUE.

1.2.4 Principio central: intención vs. implementación

El DSL declara *qué* debe ocurrir (“colorear los profesores con conflicto de horario”); el generador decide *cómo* implementarlo según el destino (para el backend Excel, genera una (FormulaRule SUMPRODUCT^{Q7}) (que persiste y recalcula en la hoja^{Q8})). El programador del DSL no escribe ninguna fórmula ni ningún ((código del destino^{Q9})¹¹).

Q7: ¿Qué es esto?

Q8: ¿Qué es esto?

Q9: ¿Qué es esto?

11: :-o... no tiene nada que ver con la tesis, pero tiene nombre de novela brasileña :-o... Si la escribo y gano dinero por eso, te doy una parte 8-D.

TRANSICIÓN A LO QUE SIGUE.

1.3 Del problema a las tablas

El primer paso es decidir cómo representar el dominio del problema en filas y columnas. [Perfecto... ¿por qué? Aquí sería bueno justificar por

qué.]

1.3.1 *El problema*

[Esta no lo voy a poder escribir :-(... Arjona se me adelantó :-/.]

[Sé más específico con el nombre de la sección. Por ejemplo, para que quede claro que es *el problema de horario* para el que nos interesa construir la aplicación.]

Se necesita planificar las defensas de tesis de una facultad. Cada defensa tiene un estudiante, un tribunal de cinco profesores (tutor, oponente, presidente, vocal y secretario), un día, una hora y un local. Además, se quiere saber cuántas defensas acumula cada profesor y detectar si algún profesor tiene dos defensas programadas a la vez.

TRANSICIÓN A LO QUE SIGUE.

1.3.2 *Decidir cuántas tablas hacen falta*

Decisión de diseño

La pregunta clave es: *¿qué es una fila?*

Si una fila es **una defensa**, la tabla tiene tantas filas como defensas **se deben planificar** y sus columnas son: estudiante, los cinco roles del tribunal, día, hora y local.

Si una fila es **un profesor**, la tabla tiene tantas filas como profesores **existen** y necesita una columna calculada que cuente en cuántas defensas participa. [¿Y aquí no hay que agregar columnas para el nombre, el grado y eso?]

El libro necesita *ambas* perspectivas(÷)PUNTO. Por ese motivo se usan dos tablas en la misma hoja que se referencian entre sí.

[Te sugiero no hablar de libro, sino de aplicación.]

[¿qué es lo que se referencian entre sí? queda ambiguo]

[De los más bonitillos los cuadraditos esos :-D. Me los voy a robar para los documentos de las clases 8-).]

TRANSICIÓN A LO QUE SIGUE.

1.3.3 Tabla de defensas

[¿Y por qué estamos hablando ahora de tablas?... ¿qué pasó con las filas?]

La tabla de defensas (defensas-table) (tiene una fila por defensa¹²) :

Columna	Descripción
estudiante	Nombre del estudiante que defiende
tutor	Profesor tutor
oponente	Profesor oponente
presidente	Presidente del tribunal
vocal	Vocal del tribunal
secretario	Secretario del tribunal
dia	Día de la defensa
hora	Hora de inicio
local	Nombre del local

12: Abusador... pero si lo que me vas a poner ahora son las columnas, ¿por qué me hablas de las filas? :-/

Por qué los cinco roles van juntos y en ese orden

Los roles `tutor`, `oponente`, `presidente`, `vocal` y `secretario` son columnas consecutivas. Esta disposición permite referenciarlos como un rango continuo^a (`trange defensas-table tutor secretario`) al calcular cuántas veces aparece un profesor en cualquier rol, sin necesidad de sumar cinco búsquedas separadas.^b

^a¿Qué es un rango continuo?

^b¿Por qué hay que sumar 5?

TRANSICIÓN A LO QUE SIGUE.

[¿Y ya creamos la tabla de las defensas? ¿por qué me hablas de ella si no la vamos a crear ahora?]

1.3.4 Tabla de profesores

La tabla de profesores (profesores-table) (tiene una fila por profesor:¹³)

13: Y dale con las filas... para después hablar de columnas.

Columna	Descripción
nombre	Nombre del profesor
grado	Grado académico
defensas	Número de defensas en que participa (calculada)

[¿Y ya? ¿se acabó esta sección? :-o... ¡qué falta de personalidad tiene esta sección! :-/. Sácale más partido.]

TRANSICIÓN A LO QUE SIGUE.

1.4 Definir la tabla de defensas

[Brutal empezar la sección con el código...]

Sintaxis: def-table

```
1 (def-table nombre-tabla
2   ((col1 "Cabecera1") (col2 "Cabecera2") ...))
3   :cell-height N
4   :cell-width N)
```

Cada par ((columna *Cabecera*)¹⁴) declara (el nombre interno^{Q10}) (símbolo DSL) y (el texto del encabezado visual de la columna^{Q11}). Las opciones :cell-height y :cell-width controlan cuántas filas y columnas físicas (de Excel¹⁵) ocupa cada (fila o columna lógica^{Q12}) (el valor 1 es el caso habitual).

TRANSICIÓN A LA TABLA QUE SIGUE.

Listing 1.1: Definición de defensas-table.

```
1 (def-table defensas-table
2   ((estudiante "") (tutor "") (oponente "") (
3     presidente ""))
4   (vocal "") (secretario "") (dia "") (hora "") (
5     local ""))
6   :cell-height 1
7   :cell-width 1)
```

(El valor en cada par indica que la columna se rellena con datos externos¹⁶) ; (no es un encabezado visible^{Q13}) sino el valor por defecto de la celda antes de que el generador coloque las fórmulas.

14: revisa que cabecera aparece con un chirimbo

Q10: ¿Qué es esto?

Q11: ¿Qué es esto?

15: ¿no habíamos quedado en que esta primera parte era agnóstica al backend?

Q12: ¿Qué es esto?

16: Aquí te propondría usar otro símbolo para eso, que en definitiva los símbolos nos salen gratis.

Q13: ¿Qué es esto?

[Me pasa lo mismo que en la versión anterior... deja esta tabla para lo último... después que tengas las demás cosas definidas.]

[Y otra cosa... ¿eso no tengo que agregarlo a algún fichero?]

TRANSICIÓN A LO QUE SIGUE.

1.5 Definir la tabla de profesores

La tabla de profesores introduce tres características nuevas: columnas calculadas, referencias cruzadas y formato condicional. [Justifica aquí por qué esa tabla necesita esas funcionalidades.](#)

TRANSICIÓN A LO QUE SIGUE.

1.5.1 Columna calculada con `:computed`

La opción `:computed` asocia columnas a (expresiones DSL^{Q14}) (que el generador traduce a fórmulas Excel¹⁷). El valor de esa celda en los datos de entrada se ignora: la celda mostrará siempre el resultado de la fórmula.

Q14: ¿Qué es esto?

17: Esto es demasiado específico

[Pon un ejemplo con palabras antes de poner el código.](#)

TRANSICIÓN AL CÓDIGO.

Listing 1.2: Columna defensas calculada con COUNTIF.

```
1 (def-table profesores-table
2   ((nombre "") (grado "") (defensas ""))
3   :cell-height 1
4   :cell-width 1
5   :computed
6   ((defensas (countif (trange defensas-table tutor
7                        secretario)
                        (col nombre)))))
```

La expresión `(countif rango criterio)` (genera la fórmula Excel COUNTIF(rango, criterio)¹⁸).

TRANSICIÓN A LO QUE SIGUE.

18: Demasiado específico de Excel, esta parte del tutorial debería ser genérica, para que se pueda implementar en cualquier backend

Cómo funciona trange

(trange defensas-table tutor secretario) genera un rango^a que abarca desde la columna `tutor` hasta la columna `secretario`^b de `defensas-table`, para todas las filas de datos. En Excel^c se traduce a algo como `$B$4:$F$6`.

La expresión completa^d cuenta cuántas veces aparece el nombre del profesor actual (tomado de su propia columna `nombre` con (col nombre)) en cualquiera de los cinco roles de cualquier defensa.

Se usa `trange` en lugar de `range`^e porque ambas tablas tienen una columna llamada `nombre`: `trange` especifica explícitamente la tabla de origen y evita la ambigüedad. **[¿Y entonces cuál es la diferencia conceptual entre trange y range?]**

^a[¿qué es un rango?]

^b[¿Y si quisiera buscar en varias columnas que no sean consecutivas?]

^c[Eso es demasiado específico para este momento del tutorial. Deberías explicar el funcionamiento de manera genérica]

^d[¿qué expresión completa? Aquí solo estás hablando del trange]

^e[¿quéjeto? :-o]

TRANSICIÓN A LO QUE SIGUE.

1.5.2 Formato condicional con `:render`

La opción `:render` acepta una forma (**conditional-rendering**^{Q15}) que especifica bajo qué condición se colorean determinadas columnas. La condición se expresa con el cuantificador `exists`.

Q15: ¿Qué es esto?

Primero pon un ejemplo con palabras... antes del código.

Listing 1.3: Formato condicional sobre profesores-table.

```
1 (def-table profesores-table
2   ((nombre "") (grado "") (defensas ""))
3   :cell-height 1
4   :cell-width 1
5   :computed
6   ((defensas (countif (trange defensas-table tutor
7                           secretario)
8                           (col nombre))))
9   :render
10  (conditional-rendering
   :condition
```

```

11      (exists (r :rows-of defensas-table)
12        :self-in (tutor oponente presidente vocal
13          secretario)
14        :matching (dia hora))
      :target-columns (nombre))

```

La variable `r` es (un alias documental^{Q16}) de la (fila que se comprueba^{Q17}), análogo a un alias de tabla en SQL.

Q16: ¿Qué es esto?

Q17: ¿Qué es esto?

Ejemplo aquí... que no entiendo mucho...

Sintaxis: exists :rows-of

```

1  (exists (var :rows-of tabla-id)
2    :self-in (col1 col2 ...)
3    :matching (clave1 clave2 ...))

```

Parámetro	Significado
:rows-of	Tabla en la que se busca
:self-in	Lista de columnas de esa tabla donde debe aparecer el valor de la celda actual
:matching	Columnas cuyo slot debe tener más de una defensa programada (conflicto)

La semántica de la condición del ejemplo es: “existe alguna fila de `defensas-table` en la que el nombre de este profesor aparece en alguno de los cinco roles, y cuyo par (dia, hora) tiene más de una defensa programada”.

El coloreado persiste en Excel

[Esto es demasiado específico de excel... en este punto del tutorial esos detalles no importan... de hecho, si la persona no sabe excel, no sabe qué está mirando aquí. Y hay muuuuchas personas que no saben excel :-(.]

El generador no colorea celdas directamente: incrusta una `FormulaRule` en el archivo `.xlsx`. Esta regla se reevalúa automáticamente cada vez que el usuario edita los datos en Excel, sin necesidad de volver a ejecutar el DSL.

La fórmula generada para este ejemplo es del tipo:

```
=SUMPRODUCT (
```

TRANSICIÓN A LO QUE SIGUE.

1.7 Construir el libro

Con las tablas declaradas y los datos preparados, se ensambla el (workbook^{Q19}) con tres macros: libro, hoja y tabla.

Q19: ¿Qué es esto?

Sintaxis: libro / hoja / tabla

```
1 (libro nombre-workbook
2   :filename "nombre.xlsx"
3   :hojas (list
4     (hoja "Nombre hoja"
5       (tabla plantilla1 :data *DATOS-1*)
6       (tabla plantilla2 :data *DATOS-2*)
7       :fixed-expressions
8         (((nav ancla dir N) (expresion))
9          ...))))
```

La macro tabla instancia (una plantilla^{Q20}) (def-table) con datos concretos. (Cuando una hoja recibe más de una tabla, el generador las coloca lado a lado dentro de la misma (región horizontal^{Q21}), separadas por una columna en blanco.²²)

Q20: ¿Qué es esto?

Q21: ¿Qué es esto?

La clave :fixed-expressions coloca expresiones o etiquetas fuera de las tablas. La posición se especifica con nav, (que calcula la celda²³) a partir de la última fila de una tabla más una serie de pasos direccionales.

22: Esto es un detalle del backend de excel... no es relevante todavía.

23: esto es solo relevante para excel... en emacs no existe esa posibilidad porque no todo son celdas así que no estoy seguro de que debería estar en el lenguaje...

Listing 1.6: Workbook completo con totales de pie de tabla.

```
1 (libro horario-tesis
2   :filename "Horario_Tesis.xlsx"
3   :hojas (list
4     (hoja "Defensas"
5       (tabla defensas-table :data *DATOS-DEFENSAS*)
6       (tabla profesores-table :data *DATOS-PROFESORES
7         *)
8       :fixed-expressions
9         (((nav (ultima-fila defensas-table estudiante
10              ) abajo 1)
11          (str "Total defensas:"))
12         ((nav (ultima-fila defensas-table estudiante
13              ) abajo 1 derecha 1)
14          (counta (trange defensas-table estudiante))
15          )
16         ((nav (ultima-fila profesores-table nombre)
```

```

13         abajo 1)
14         (str "Total profesores:"))
15         ((nav (ultima-fila profesores-table nombre)
                abajo 1 derecha 2)
            (sum-range (trange profesores-table
                           defensas))))))

```

[Aquí horario sigue teniendo un filename... eso me tiene muy nervioso...]

TRANSICIÓN A LO QUE SIGUE.

Sintaxis: nav / ultima-fila

```

1  ;; Posicion relativa a la ultima fila de una
   columna:
2  (nav (ultima-fila tabla-id columna) abajo N)
3  (nav (ultima-fila tabla-id columna) abajo N
        derecha M)
4  (nav (primera-fila tabla-id columna) arriba N)

```

[Recuerda que en otros contextos esto no tiene por qué existir, tanto lo que está en el bloque anterior como en el que sigue.]

Por qué usar nav en lugar de coordenadas fijas

Las coordenadas absolutas de la celda “una fila debajo de la última defensa” dependen de cuántas defensas haya en los datos. nav las calcula automáticamente en tiempo de generación, de modo que si se añaden más filas de datos los totales se desplazan solos sin modificar el programa DSL^a.

^a[¿qué es el programa DSL?]

TRANSICIÓN A LO QUE SIGUE.

1.8 Generar y ejecutar

Las dos últimas líneas del programa desencadenan la generación del archivo:

Listing 1.7: Compilacion y ejecucion desde el programa DSL.

```

1 (xl-generate horario-tesis "horario-tesis.py")
2 (xl-run-generated "horario-tesis.py")

```

`xl-generate` recorre el AST del workbook y produce el programa de construcción del libro. `xl-run-generated` ejecuta ese programa con `python3`; el resultado es el archivo `Horario_Tesis.xlsx` en disco.

El archivo `.xlsx` contiene tanto las fórmulas de celda como las `FormulaRule` de formato condicional; estas últimas permanecen activas cuando el usuario abre y edita el libro directamente en Excel.

1.8.1 Ejecución desde la terminal con el script `generar.sh`

Como alternativa a invocar SBCL directamente, el directorio incluye un script Bash `generar.sh` que automatiza el mismo flujo:

```
# Primera vez: dar permisos de ejecucion
chmod +x generar.sh

# Ejecutar el flujo completo
./generar.sh
```

El script realiza exactamente los mismos tres pasos que ocurren al cargar el programa DSL a mano:

1. Sitúa el directorio de trabajo en la carpeta del tutorial (para que los `load` relativos del programa DSL funcionen independientemente del directorio desde el que se llame al script).
2. Invoca `sbcl --script tutorial-horario.lisp`, que expande las macros, construye el AST, genera `horario-tesis.py` y lo ejecuta con `python3`.
3. Verifica que `Horario_Tesis.xlsx` fue creado e imprime su tamaño.

¿Cuándo usar el script y cuándo cargar directamente?

El script es conveniente para ejecutar el tutorial de un solo paso desde cualquier directorio. Cargar el archivo directamente desde el REPL de SBCL (`(load "tutorial-horario.lisp")`) es preferible durante el desarrollo, porque permite inspeccionar el estado del AST en memoria entre las distintas formas del DSL.

1.9 El programa completo

A continuación se presenta el contenido íntegro de los dos archivos que conforman el programa.

1.9.1 *data-tutorial.lisp*

Listing 1.8: data-tutorial.lisp — datos de entrada.

```
1 (defparameter *DATOS-DEFENSAS*
2   '(((" " " " " " " " " " " " " ")
3     (" " " " " " " " " " " " " ")
4     ("Estudiante" "Tutor" "Oponente" "Presidente" "
5       Vocal"
6       "Secretario" "Dia" "Hora" "Local")
7     ("Juan Diaz" "Piad" "Garcia" "Lopez" "Torres"
8       "Ruiz"
9       "Lunes" "10:00" "
10      Aula 1")
11     ("Maria Vega" "Garcia" "Lopez" "Piad" "Soto"
12       "Torres"
13       "Lunes" "14:00" "
14      Aula 2")
15     ("Luis Mora" "Lopez" "Torres" "Garcia" "Ruiz"
16       "Soto"
17       "Martes" "10:00" "
18      Aula 1"))))
19
20 (defparameter *DATOS-PROFESORES*
21   '(((" " " " " ")
22     (" " " " " ")
23     ("Nombre" "Grado" "Defensas")
24     ("Piad" "Dr" " ")
25     ("Garcia" "Dr" " ")
26     ("Lopez" "Msc" " ")
27     ("Torres" "Msc" " ")
28     ("Ruiz" "Lic" " ")
29     ("Soto" "Msc" " ")))
```

1.9.2 *tutorial-horario.lisp*

Listing 1.9: tutorial-horario.lisp — programa DSL completo.

```

1 (load "dsl-directo.lisp")
2 (load "data-tutorial.lisp")
3
4 ;; TABLA 1: defensas
5 ;; Nueve columnas: cinco roles consecutivos (tutor...
6   secretario),
7 ;; dia, hora, local. Sin coloreado propio.
8 (def-table defensas-table
9   ((estudiante "") (tutor "") (oponente "") (
10    presidente ""))
11   (vocal "") (secretario "") (dia "") (hora "") (
12    local ""))
13   :cell-height 1
14   :cell-width 1)
15
16 ;; TABLA 2: profesores
17 ;; Columna "defensas" calculada con COUNTIF.
18 ;; Coloreado condicional: nombre en rojo si el
19   profesor tiene conflicto.
20 (def-table profesores-table
21   ((nombre "") (grado "") (defensas ""))
22   :cell-height 1
23   :cell-width 1
24   :computed
25   ((defensas (countif (trange defensas-table tutor
26    secretario)
27    (col nombre))))
28   :render
29   (conditional-rendering
30    :condition
31    (exists (r :rows-of defensas-table)
32     :self-in (tutor oponente presidente vocal
33     secretario)
34     :matching (dia hora))
35     :target-columns (nombre)))
36
37 ;; WORKBOOK
38 (libro horario-tesis
39   :filename "Horario_Tesis.xlsx"
40   :hojas (list
41    (hoja "Defensas"
42     (tabla defensas-table :data *DATOS-DEFENSAS*)

```

```

37      (tabla profesores-table :data *DATOS-PROFESORES
38        *)
39      :fixed-expressions
40      (((nav (ultima-fila defensas-table estudiante
41              ) abajo 1)
42        (str "Total defensas:"))
43      ((nav (ultima-fila defensas-table estudiante
44              ) abajo 1 derecha 1)
45        (counta (trange defensas-table estudiante))
46        )
47      ((nav (ultima-fila profesores-table nombre)
48              abajo 1)
49        (str "Total profesores:"))
50      ((nav (ultima-fila profesores-table nombre)
51              abajo 1 derecha 2)
52        (sum-range (trange profesores-table
53                    defensas))))))
54
55 (xl-generate horario-tesis "horario-tesis.py")
56 (xl-run-generated "horario-tesis.py")

```

1.10 Resultado esperado

El programa genera Horario_Tesis.xlsx con la siguiente estructura:

Rango	Contenido
Columnas A–I, filas 4–6	defensas-table: tres defensas
Columna J	Separador en blanco
Columnas K–M, filas 4–9	profesores-table: seis profesores
Columna M, filas 4–9	Fórmulas COUNTIF generadas
Fila 7 (cols. A–B)	Totales de defensas (COUNTA)
Fila 11 (cols. K, M)	Totales de profesores (SUM)
Rango K4:K9	FormulaRule: nombres en rojo si hay conflicto

Con los datos de ejemplo, las defensas de “Juan Díaz” y “Luis Mora” comparten el slot **Lunes 10:00** con roles solapados, por lo que los profesores Piad, García, López, Torres, Ruiz y Soto aparecen resaltados en rojo.

1.11 Traza del diseño

El cuadro siguiente resume las decisiones tomadas durante el modelado, relacionando cada requisito del problema con su representación en el DSL.

Requisito	Representación	Decisión de diseño
Una fila por defensa con tribunal completo	<code>defensas-table</code>	Granularidad natural; agrupa todas las columnas de tribunal
Los cinco roles en columnas consecutivas	<code>orden</code> en <code>def-table</code>	Permite <code>trange</code> tutor <code>secretario</code> para contar roles
Contar defensas por profesor	<code>columna</code> <code>computed</code>	<code>countif</code> sobre el rango de roles de <code>defensas-table</code>
Detectar profesores con conflicto	<code>:render</code> con <code>exists</code> <code>:rows-of</code>	El backend genera una <code>FormulaRule</code> que persiste en Excel
Totales dinámicos al pie de tabla	<code>:fixed-expressions</code> con <code>nav</code>	Las coordenadas se recalculan si cambia el número de filas

1.12 Referencia rápida de formas

Forma	Descripción
<code>(def-table nom cols opts...)</code>	Define una plantilla de tabla
<code>:computed ((col expr))</code>	Columna calculada con fórmula Excel
<code>:render (conditional-rendering)</code>	Regla de coloreado condicional
<code>(conditional-rendering :condition :target-columns)</code>	Asocia condición con columnas a colorear
<code>(exists (r :rows-of tabla) ...)</code>	Existencia en otra tabla (cross-table)
<code>(exists (r :all-rows) ...)</code>	Existencia en la misma tabla
<code>(col nombre)</code>	Valor de la columna en la fila actual
<code>(trange tabla desde hasta)</code>	Rango en otra tabla
<code>(countif rango criterio)</code>	Fórmula COUNTIF
<code>(counta rango)</code>	Fórmula COUNTA
<code>(sum-range rango)</code>	Fórmula SUM
<code>(str "texto")</code>	Literal de texto en celda
<code>(nav (ultima-fila t c) dir N)</code>	Posición relativa a última fila
<code>(nav (primera-fila t c) dir N)</code>	Posición relativa a primera fila
<code>(libro nom :filename :hojas)</code>	Crea el workbook
<code>(hoja nom tablas :fixed-expressions)</code>	Define una hoja del libro
<code>(tabla plantilla :data datos)</code>	Instancia una plantilla con datos
<code>(xl-generate wb archivo)</code>	Genera el programa de construcción
<code>(xl-run-generated archivo)</code>	Ejecuta el programa y crea el .xlsx

1.13 Formas del DSL no utilizadas en este tutorial

El tutorial cubre un subconjunto del DSL suficiente para construir el horario de defensas. Las formas que se describen a continuación están disponibles y resultan útiles en escenarios más complejos.

Condicionales en celdas

Permiten que el valor de una columna dependa de una condición evaluada fila a fila.

Forma	Para qué sirve
<code>(_if cond entonces sino)</code>	IF de tres ramas. Muestra un valor sólo cuando se cumple la condición; en caso contrario muestra otro.
<code>(non-empty expr)</code>	Verdad si la expresión no está vacía (<code><></code>). Útil como condición de <code>_if</code> .
<code>(show-nothing)</code>	Produce la cadena vacía <code>.</code> Se usa como rama <i>sino</i> para dejar la celda en blanco.
<code>(equals a b) / (different a b)</code>	Igualdad y desigualdad entre dos expresiones.
<code>(_and a b) / (_or a b)</code>	Conjunción y disyunción lógica.

Ejemplo: mostrar el nombre de una sala sólo si la fila tiene un evento asignado, y dejarla en blanco en caso contrario.

```
1 (col "Sala" :computed
2   (_if (non-empty (col "Evento"))
3     (col "Sala")
4     (show-nothing)))
```

Navegación entre filas

Permiten que una columna acceda al valor de la misma celda en la fila anterior o siguiente, lo que es indispensable para calcular diferencias temporales entre registros consecutivos.

Forma	Para qué sirve
(previous-row expr)	Desplaza la referencia una fila hacia arriba.
(previous-of base)	Valor de base en la fila inmediatamente anterior.
(next-of base)	Valor de base en la fila siguiente.
(it-is-the-first-row)	Verdad en la primera fila de datos. Se combina con <code>_if</code> para el caso borde donde no existe fila anterior.
<code>primera-fila</code>	Ancla de posición a la primera fila de la tabla; complemento de <code>ultima-fila</code> .

Ejemplo: calcular la duración de cada franja horaria como la diferencia entre su hora de fin y la hora de fin de la fila anterior.

```

1 (col "Duracion" :computed
2   (_if (it-is-the-first-row)
3     (str "")
4     (subtract (col "Hora fin")
5               (previous-of (col "Hora fin")))))

```

Aritmética y texto

Forma	Para qué sirve
(add a b) / (subtract a b)	Suma y resta sobre valores numéricos o de celda.
(multiply a b) / (divide a b)	Multiplicación y división.
(time-add hora minutos)	Suma una cantidad de minutos a una hora Excel (<code>hora + minutos/1440</code>).
(concat a b)	Concatenación de texto: combina el contenido de dos expresiones en una sola cadena.

Ejemplo: calcular la hora de fin a partir de la hora de inicio y una duración fija en minutos.

```

1 (col "Hora fin" :computed
2   (time-add (col "Hora inicio") (col "Duracion min")))

```

Referencias avanzadas entre tablas

Forma	Para qué sirve
(tcol tabla nombre)	Como col, pero especifica explícitamente de qué tabla proviene la columna. Necesario cuando dos tablas de la misma hoja comparten un nombre de columna y hay que desambiguar.
(range desde hasta)	Rango de columnas de la tabla <i>actual</i> , sin necesidad de nombrarla. Equivalente de <code>trange</code> para la tabla en curso.
(lookup clave campo)	Búsqueda estilo VLOOKUP: dado el valor de <code>clave</code> en la fila actual, devuelve el valor de <code>campo</code> en la tabla de referencia.

Plantillas paramétricas

Cuando una misma estructura de tabla debe instanciarse varias veces con un detalle variable —por ejemplo, la misma tabla de franjas para cada día de la semana— se declaran *inst-params* en `def-table`:

```
1 (def-table franja-dia
2   ((turno "") (salon "") (grupo ""))
3   :inst-params (dia)
4   :computed
5   ((salon (tcol sala-disponible salon))))
```

Dentro de la definición, `(param nombre)` accede al valor que se pase al instanciar. Al construir la hoja se indica el parámetro como argumento con nombre:

```
1 (hoja "Semana"
2   (tabla franja-dia :dia 'lunes :data *datos-lunes*)
3   (tabla franja-dia :dia 'martes :data *datos-martes
4     *)))
```

Con este mecanismo el DSL genera cinco hojas con estructura idéntica y fórmulas parametrizadas con el nombre del día, a partir de una sola declaración `def-table`.

Detección de solapamiento dentro de la misma tabla

El tutorial usa `(exists (r :rows-of tabla) ...)` para buscar conflictos en una tabla *distinta*. La variante `:all-rows` detecta solapamientos dentro de la *propia* tabla:

```
1 (conditional-rendering
2   (exists (r :all-rows)
3     :matching (:col-a "Inicio" :col-b "Fin")
4     :any-overlap t)
5   :format '(:bg "#FF9999"))
```

Útil, por ejemplo, para marcar franjas que coinciden en la misma tabla de turnos de un único recurso.

Celdas de varias filas o columnas

Las opciones `:cell-height N` y `:cell-width N` de `def-table` hacen que cada fila lógica ocupe N filas (o columnas) físicas en el Excel generado. Es adecuado para horarios donde cada franja horaria debe abarcar varias celdas de la cuadrícula, o para encabezados que fusionan columnas.

```
1 (def-table horario-visual
2   ((hora "") (actividad "") (sala ""))
3   :cell-height 2) ; cada franja ocupa 2 filas
                     físicas
```

Hojas con tablas apiladas verticalmente

La macro `hoja-v` es la versión vertical de `hoja`: apila las tablas una encima de otra en lugar de colocarlas en columnas. Se usa cuando las tablas tienen muchas columnas y el libro se orienta para impresión en apaisado, o cuando las tablas representan secciones consecutivas de un mismo listado.

```
1 (hoja-v "Resumen"
2   (tabla tabla-manana :data *datos-manana*)
3   (tabla tabla-tarde :data *datos-tarde*))
```

Referencias cruzadas entre hojas

Para libros con varias hojas que necesitan compartir datos, el DSL ofrece dos formas:

Forma	Para qué sirve
<code>(sheet-ref hoja plantilla-celda)</code>	Genera una referencia a una celda de otra hoja, expresada como plantilla de texto en la que <code>\${row-num}</code> se sustituye por el número de fila real.
<code>(cross-cell :sheet var :col col :row fila)</code>	Referencia a una celda concreta de otra hoja, identificada por su variable de hoja, columna y fila. Se usa cuando es necesario leer un campo de cada hoja en un bucle de generación.